
Manual de Utilização da API

Postagens e Comentários · REST · JSON

Guia destinado a desenvolvedores júnior
Node.js >= 20.6 · Sem dependências externas

Sumário

1. Visão geral
2. Requisitos e preparação do ambiente
3. Como executar a API
4. Variáveis de ambiente (.env.local)
5. Endpoints disponíveis
6. Exemplos de uso passo a passo
7. Filtros de consulta de comentários
8. Códigos de resposta e tratamento de erros
9. Dúvidas frequentes

1. Visão geral

Esta é uma API REST para cadastro de postagens e comentários. Ela recebe e devolve dados no formato JSON, permitindo:

- Cadastrar postagens (posts);
- Inserir comentários em uma postagem;
- Consultar todas as postagens ou uma postagem específica;
- Consultar todos os comentários, os comentários de uma postagem, ou filtrar comentários por autor, conteúdo ou postagem;
- Remover uma postagem (e, em cascata, todos os seus comentários);
- Remover um comentário específico;
- Gerar sugestões de postagem com IA a partir de uma ideia (agente Mastra com `gpt-4o-mini`).

A persistência é feita em um arquivo JSON local (`data/db.json`). Não é necessário instalar banco de dados. O projeto usa apenas o módulo nativo `node:http` do Node.js, sem dependências externas.

Dica As variáveis de ambiente ficam no arquivo `.env.local`. Nunca commite esse arquivo em um repositório público.

2. Requisitos e preparação do ambiente

2.1 Node.js

- Node.js versão 20.6 ou superior (preferencialmente a versão LTS).
- `npm` já vem junto com o Node.js.

Verifique sua instalação:

```
node --version
npm --version
```

2.2 Estrutura do projeto

```
.
├── index.js           # Servidor HTTP e rotas
├── lib/
│   └── store.js       # Camada de persistência (JSON file)
├── .env.local         # Variáveis de ambiente
├── .env.example       # Modelo das variáveis
└── package.json       # Scripts npm
```

Observação Os arquivos `data/*.json` são criados automaticamente na primeira execução. O diretório `data/` e o arquivo `.env.local` estão ignorados no `.gitignore`.

3. Como executar a API

1. Acesse a pasta do projeto:

```
cd rocketseat-blog-ia-nodejs
```

2. Instale as dependências (o projeto não possui dependências externas neste caso, mas o comando é seguro):

```
npm install
```

3. Inicie o servidor:

```
npm start
```

Você deve ver a mensagem:

```
API rodando em http://0.0.0.0:3000
```

Dica Use `npm run dev` durante o desenvolvimento: o servidor reinicia automaticamente quando o código é alterado (modo watch).

3.1 Testar rapidamente se a API está no ar

```
curl http://localhost:3000/api/health
```

Resposta esperada:

```
{
  "status": "ok"
}
```

4. Variáveis de ambiente (.env.local)

O arquivo `.env.local` é carregado na inicialização via `node --env-file=.env.local` (não é necessária a biblioteca `dotenv`).

Variável	Padrão	Descrição
PORT	3000	Porta em que a API escutará requisições.
HOST	0.0.0.0	Endereço de escuta (0.0.0.0 = aceita conexões externas).
DB_FILE	./data/db.json	Caminho do arquivo onde os dados são persistidos.
OPENAI_API_KEY	—	Chave da OpenAI. Necessária para o agente Mastra gerar sugestões de postagem.

Exemplo de conteúdo do `.env.local`:

```
PORT=3000
HOST=0.0.0.0
DB_FILE=./data/db.json
OPENAI_API_KEY=sk-sua-chave-aqui
```

Atenção O arquivo `.env.local` não deve ser enviado a sistemas de versionamento. Use o `.env.example` como modelo para divulgar quais variáveis existem.

5. Endpoints disponíveis

Resumo de todas as rotas da API (prefixo `/api`):

Método	Rota	Descrição
GET	<code>/api/health</code>	Verifica se a API está no ar.
POST	<code>/api/posts</code>	Cadastra uma postagem.
GET	<code>/api/posts</code>	Lista todas as postagens.
GET	<code>/api/posts/:id</code>	Consulta uma postagem pelo id.
DELETE	<code>/api/posts/:id</code>	Remove uma postagem e, em cascata, todos os seus comentários.
POST	<code>/api/posts/:id/comments</code>	Insere um comentário em uma postagem.
GET	<code>/api/posts/:id/comments</code>	Lista os comentários de uma postagem.
GET	<code>/api/comments</code>	Lista todos os comentários ou filtra por parâmetros de consulta.
DELETE	<code>/api/comments/:id</code>	Remove um comentário específico.
GET	<code>/api/suggestions</code>	Lista as sugestões de postagem geradas pelo agente.
POST	<code>/api/suggestions</code>	Gera uma sugestão de postagem a partir da ideia enviada em <code>body.idea</code> .

6. Exemplos de uso passo a passo

6.1 Cadastrar uma postagem

POST `/api/posts`

Campos obrigatórios: `title` e `content`. Opcional: `author` (assume "Anônimo" se ausente).

```
curl -X POST http://localhost:3000/api/posts \
-H "Content-Type: application/json" \
-d '{
  "title": "Introdução ao Node.js",
  "content": "Neste artigo vamos falar sobre o básico do Node.js.",
  "author": "Ana"
}'
```

Resposta (201 Created):

```
{
  "post": {
    "id": "5868b233-4b12-41f7-b2f0-420b3c83e9d9",
    "title": "Introdução ao Node.js",
    "content": "Neste artigo vamos falar sobre o básico do Node.js.",
    "author": "Ana",
    "createdAt": "2026-09-16T21:27:32.526Z",
    "updatedAt": "2026-09-16T21:27:32.526Z"
  }
}
```

Guarde o valor do `id`: ele será usado nas próximas requisições.

6.2 Listar todas as postagens

GET `/api/posts`

```
curl http://localhost:3000/api/posts
```

6.3 Consultar uma postagem específica

GET `/api/posts/:id`

```
curl http://localhost:3000/api/posts/5868b233-4b12-41f7-b2f0-420b3c83e9d9
```

Se o id não existir, a API responde 404 com:

```
{ "error": "Postagem não encontrada" }
```

6.4 Inserir um comentário

POST `/api/posts/:id/comments`

Campo obrigatório: `content`. Opcional: `author` (assume "Anônimo").

```
curl -X POST http://localhost:3000/api/posts/5868b233-4b12-41f7-b2f0-420b3c83e9d9/comments \
-H "Content-Type: application/json" \
-d '{
  "content": "Ótimo artigo, aprendi muito!",
  "author": "Carlos"
}'
```

Resposta (201 Created):

```
{
  "comment": {
    "id": "335d6e65-d38a-4a28-9750-abf4f0a74b59",
    "postId": "5868b233-4b12-41f7-b2f0-420b3c83e9d9",
    "content": "Ótimo artigo, aprendi muito!",
    "author": "Carlos",
    "createdAt": "2026-09-16T21:27:34.230Z"
  }
}
```

6.5 Listar comentários de uma postagem

GET `/api/posts/:id/comments`

```
curl http://localhost:3000/api/posts/5868b233-4b12-41f7-b2f0-420b3c83e9d9/comments
```

Resposta:

```
{
  "comments": [
    {
      "id": "335d6e65-d38a-4a28-9750-abf4f0a74b59",
      "postId": "5868b233-4b12-41f7-b2f0-420b3c83e9d9",
      "content": "Ótimo artigo, aprendi muito!",
      "author": "Carlos",
      "createdAt": "2026-09-16T21:27:34.230Z"
    }
  ],
  "total": 1
}
```

6.6 Remover uma postagem

DELETE /api/posts/:id

Remove a postagem informada e, **em cascata**, todos os comentários vinculados a ela. Os comentários da postagem também deixam de aparecer em `GET /api/comments` e `GET /api/posts/:id/comments`.

```
curl -X DELETE http://localhost:3000/api/posts/5868b233-4b12-41f7-b2f0-420b3c83e9d9
```

Resposta (200 OK) — devolve a postagem removida:

```
{
  "post": {
    "id": "5868b233-4b12-41f7-b2f0-420b3c83e9d9",
    "title": "Introdução ao Node.js",
    "content": "Neste artigo vamos falar sobre o básico do Node.js.",
    "author": "Ana",
    "createdAt": "2026-09-16T21:27:32.526Z",
    "updatedAt": "2026-09-16T21:27:32.526Z"
  }
}
```

Se o id não existir, a API responde 404 com:

```
{ "error": "Postagem não encontrada" }
```

6.7 Remover um comentário

DELETE /api/comments/:id

Remove um comentário específico. Não é preciso informar a postagem.

```
curl -X DELETE http://localhost:3000/api/comments/335d6e65-d38a-4a28-9750-abf4f0a74b59
```

Resposta (200 OK) — devolve o comentário removido:

```
{
  "comment": {
    "id": "335d6e65-d38a-4a28-9750-abf4f0a74b59",
    "postId": "5868b233-4b12-41f7-b2f0-420b3c83e9d9",
    "content": "Ótimo artigo, aprendi muito!",
    "author": "Carlos",
    "createdAt": "2026-09-16T21:27:34.230Z"
  }
}
```

Se o id não existir, a API responde 404 com:

```
{ "error": "Comentário não encontrado" }
```

6.8 Gerar uma postagem com inteligência artificial

POST `/api/suggestions`

O agente Mastra (`openai/gpt-4o-mini`) recebe a ideia no campo `idea` e cria uma sugestão de postagem completa em Markdown. Requer a variável `OPENAI_API_KEY` configurada no `.env.local`.

```
curl -X POST http://localhost:3000/api/suggestions \
  -H "Content-Type: application/json" \
  -d '{
    "idea": "Um guia para iniciantes em Node.js"
  }'
```

Resposta (201 Created):

```
{
  "suggestion": {
    "id": "mP9xQ2vD8fK2wZ7rLc4nB",
    "title": "Node.js para Iniciantes: Primeiros Passos",
    "content": "# Node.js para Iniciantes\n\nNeste guia você aprende...",
    "published_at": null,
    "created_at": "2026-09-17T21:30:00.000Z",
    "approved_at": null,
    "rejected_at": null
  }
}
```

Guarde o `id`: a sugestão nasce pendente de aprovação — `approved_at` e `rejected_at` ficam `null` até serem decididos, assim como `published_at` antes da publicação.

Se o campo `idea` faltar, a API responde 400 com `{ "error": "O campo 'idea' é obrigatório" }`. Sem `OPENAI_API_KEY` configurada, responde 500 com erro informando a chave ausente.

7. Filtros de consulta de comentários

O endpoint GET `/api/comments` lista todos os comentários caso nenhum parâmetro seja informado, ou filtra o resultado por meio de query parameters:

Parâmetro	Descrição	Exemplo
<code>postId</code>	Comentários de uma postagem específica (igualdade).	<code>/api/comments?postId=<id></code>
<code>author</code>	Comentários cujo autor contém o texto (case-insensitive).	<code>/api/comments?author=maria</code>
<code>content</code>	Comentários cujo conteúdo contém o texto (case-insensitive).	<code>/api/comments?content=artigo</code>

Observação Os filtros podem ser combinados entre si, por exemplo: `/api/comments?postId=...&author=ana`.

7.1 Exemplos

```
# Todos os comentários
curl "http://localhost:3000/api/comments"

# Filtro por autor
curl "http://localhost:3000/api/comments?author=maria"

# Filtro por conteúdo
curl --get "http://localhost:3000/api/comments" --data-urlencode "content=ótimo"

# Filtro por postagem
curl "http://localhost:3000/api/comments?postId=5868b233-4b12-41f7-b2f0-420b3c83e9d9"

# Filtros combinados
curl "http://localhost:3000/api/comments?postId=5868b233-4b12-41f7-b2f0-420b3c83e9d9&author=ana"
```

Atenção: ao usar acentos ou espaços em filtros, utilize a codificação de URL (ex.: o caractere `ó` vira `%C3%B3`). O curl com `--data-urlencode` faz essa codificação automaticamente.

8. Códigos de resposta e tratamento de erros

Código	Significado	Quando ocorre
200	OK	Consultas e remoções finalizadas com sucesso.
201	Created	Postagem ou comentário criado com sucesso.
204	No Content	Resposta ao preflight CORS (OPTIONS).
400	Bad Request	Campo obrigatório ausente, JSON inválido ou corpo muito grande.
404	Not Found	Rota inexistente, postagem ou comentário não encontrado.
405	Method Not Allowed	Método HTTP não suportado pela rota.
500	Internal Server Error	Erro interno do servidor.

Formato padrão de erro:

```
{
  "error": "Mensagem descritiva do erro"
}
```

Mensagem	Código	Causa
O campo "title" é obrigatório	400	Envios sem title ou com ele vazio.
O campo "content" é obrigatório	400	Envios sem content ou com ele vazio.
O campo "idea" é obrigatório	400	Sugestão de IA enviada sem idea ou com ele vazio.
JSON inválido no corpo da requisição	400	Corpo não é um JSON válido.
Corpo da requisição muito grande	400	Corpo acima de 1 MB.
Postagem não encontrada	404	Id da postagem não existe na base de dados.
Comentário não encontrado	404	Id do comentário não existe na base de dados.
Rota não encontrada	404	Caminho inexistente na API.
Método não permitido	405	Verbo HTTP não suportado na rota.

Dica para o consumidor da API Sempre verifique o `Content-Type: application/json` e o status HTTP. Trate os erros 400 como problemas no payload enviado e os 404 como ids/rotas inválidas.

9. Dúvidas frequentes (FAQ)

Os dados são perdidos ao reiniciar o servidor?

Não. Os dados são salvos em `data/db.json` e permanecem após reiniciar.

Preciso instalar um banco de dados?

Não. A persistência é feita em arquivo JSON, sem necessidade de banco externo.

Como mudar a porta da API?

Altere a variável `PORT` no arquivo `.env.local` e reinicie o servidor.

O projeto usa navegador para acessar?

Não. É uma API REST: consuma-a com `curl`, Postman, Insomnia ou diretamente do seu front-end via `fetch`. As respostas possuem CORS habilitado (incluindo o método `DELETE`).

Como funciona o agente de sugestões de postagem?

O endpoint `POST /api/suggestions` envia a ideia do campo `idea` para o agente Mastra, que usa o modelo `openai/gpt-4o-mini` para gerar `title` e `content` (Markdown). O registro é salvo em `data/db.json` na coleção `suggestions`, com `published_at`, `approved_at` e `rejected_at` vazios (`null`).

Posso usar acentos nos dados?

Sim. Os dados são codificados em UTF-8. Apenas lembre-se de codificar a URL quando houver acentos nos query parameters de filtro.

Exemplo de consumo com fetch (navegador / front-end)

```
// Criar uma postagem
const resposta = await fetch("http://localhost:3000/api/posts", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    title: "Meu primeiro post",
    content: "Conteúdo do post.",
    author: "Ana"
  })
});

const dados = await resposta.json();
console.log(dados.post.id);

// Remover a postagem criada
await fetch(`http://localhost:3000/api/posts/${dados.post.id}`, {
  method: "DELETE"
});
```

Surgiu um problema, o que fazer?

- Confira se o servidor está rodando (`npm start`);
- Verifique se a porta coincide com a do `.env.local`;
- Valide o JSON enviado (use um validador);
- Confirme que o Content-Type é `application/json`;
- Configure a `OPENAI_API_KEY` no `.env.local` ao usar `POST /api/suggestions`;

- Verifique se o id da postagem existe em `GET /api/posts`;
- Verifique se o id do comentário existe em `GET /api/comments`.